Deusto

ASSISTED BY AI

https://ai-label.org/



Threads

Hilos y concurrencia en Java



Disclaimer

- Para la creación de estas slides se ha utilizado texto generado con ChatGPT y Claude.
- El contenido final ha sido validado y revisado para garantizar que es correcto y está libre de errores.



Procesos, hilos y núcleos de CPU

- **Proceso**: instancia de un programa en ejecución que tiene su propio espacio de memoria.
- **Hilo (Thread)**: ***unidad lógica*** de ejecución dentro de un proceso. Los hilos comparten la memoria y recursos de un proceso.
- **Núcleo de CPU (core)**: ***unidad física*** sobre la que se ejecutan instrucciones (hilos). Los procesadores actuales tienen varios núcleos y permiten la ejecución de varios hilos en paralelo.



Ejecución secuencial, concurrente y paralela

- **Secuencial**: las tareas se ejecutan una tras otra. Hasta que termina una, no se ejecuta la siguiente.
- **Concurrente**: varias tareas progresan de forma solapada, aunque no al mismo tiempo. Se alterna entre la ejecución de distintas tareas, simulando que se ejecutan "a la vez".
- **Paralela**: varias tareas se ejecutan al mismo tiempo y de forma independiente (en varios núcleos de una CPU).



¿Qué es un Thread?

- La clase **Thread** representa un hilo de ejecución dentro de un programa o proceso.
  - *Unidad de ejecución lógica gestionada por la JVM y el sistema operativo.*
  - *Si la CPU tiene más de un núcleo, los hilos en Java se ejecutan de forma paralela; de lo contrario, se ejecutarán de forma concurrente.*



Crear e iniciar un Thread

👉 ***Ver ejemplo: ThreadBasico***

- **Heredando de Thread**
  - Crear una clase que extiende Thread y redefinir el método `run()`.
  - Crear una instancia de la nueva clase e invocar `start()`.
- **Implementando Runnable**
  - Crear una clase que implementa Runnable y definir el método `run()`.
  - Crear un Thread pasando como parámetro una instancia de la nueva clase e invocar `start()`.



Parar un Thread (1/2)

👉 ***Ver ejemplos: ThreadInterrupt1 & 2***

- **Métodos obsoletos `stop()`, `suspend()` y `resume()`**
  - No deben utilizarse: ***deprecated desde JDK 1.2***.
  - Pueden generar bloqueos y estados inconsistentes.
- **Detención ordenada (mejor práctica)**
  - Usar una variable **boolean** para indicar al hilo cuándo terminar.
  - Usar `interrupt()` + `isInterrupted()` *[siguiente slide →]*



Parar un Thread (2/2)

👉 ***Ver ejemplos: ThreadInterrupt1 & 2***

- **Método `interrupt()`**
  - Permite una terminación controlada de un hilo.
  - `interrupt()` no detiene el hilo; le indica que debe detenerse.
  - Dentro del método `run()` se revisa con `interrupted()` e `isInterrupted()`
    - Gestionar `InterruptedException` al usar `sleep()`.



Pausar y reanudar un Thread (1/3)

- **Coordinación con `wait()` / `notify()`**
  - Deben invocarse en un bloque `synchronized` y usar un objeto monitor.
  - `wait()`: el hilo libera el objeto monitor y se pone en pausa (no consume CPU) hasta que otro hilo lo despierte.
  - `notify()`: despierta **un hilo** en espera sobre un objeto monitor.
  - `notifyAll()`: despierta **los hilos** en espera sobre un objeto monitor.
  - Este mecanismo debe combinarse con una variable de control (boolean + bucle while) para evitar conflictos.



Pausar y reanudar un Thread (2/3)

```
// Bloque de pausa; dentro del bucle del método run().
synchronized (this) { // se bloquea el monitor (en este caso this)
    while (enPausa) { // gestión de reactivaciones erróneas.
        try {
            this.wait(); // se libera el monitor y se pausa el hilo
        } catch (InterruptedException e) {
            // gestión para controlar la detención mediante interrupt()
            Thread.currentThread().interrupt();
            return; // se termina la ejecución del método run()
        }
    }
}
```



Pausar y reanudar un Thread (3/3)

```
public void pausar() { // Método para pausar el hilo
    synchronized (this) {
        enPausa = true;
       // Slide previa: El hilo se pausa al entrar en el while e invocar wait().
    }
}

public void reanudar() { // Método para reanudar el hilo
    synchronized (this) {
       enPausa = false;
       this.notify();
       // Slide previa: el hilo continua la ejecución después de wait()
    }
}
```



Dormir un Thread

👉 ***Ver ejemplo: ThreadSleep***

- **Método `sleep()`**
  - Pausa la ejecución del hilo durante un tiempo fijo (miliseg.).
  - Se usa para simular esperas, retrasos o ejecución de tareas periódicas (con un cierto intervalo).
  - Si se intenta interrumpir un hilo “dormido”, se lanza una `InterruptedException`.

`join()` - Esperar a que otro hilo termine

👉 ***Ver ejemplo: ThreadJoin***

- El método `join()` permite que un hilo espere a que otro termine antes de continuar.
- Sirve para asegurar que un hilo no avance hasta que todos los hilos que inició hayan completado su trabajo.
  - *Cuando un Hilo-A invoca el método join() sobre otro Hilo-B, el Hilo-A se detiene y espera hasta que el Hilo-B termine.*



`synchronized` - Acceso concurrente

👉 ***Ver ejemplo: ThreadAccesoConcurrente***

- Un bloque `synchronized` limita el número de hilos que pueden acceder a un fragmento de código.
- Se usa para invocación de método o proteger variables y garantizar la exclusión mutua. Si varios hilos intentan acceder, hasta que termina el primero, el resto esperan.

```
public synchronized void increment() {
    count++;
}
```



Hilos en Swing y EDT

- Swing no es **thread-safe**; si no se gestionan bien los hilos, pueden surgir problemas como bloqueo de la interfaz o excepciones.
  - **Event Dispatch Thread (EDT)** es el hilo que maneja los eventos de la interfaz, (clics, movimiento de ratón) y cualquier operación de actualización de la GUI.
  - Los listeners se ejecutan en el EDT. Cualquier lógica compleja o lenta dentro de un listener puede bloquear la interfaz.
  - Las actualizaciones de la GUI, como modificar el contenido de un JLabel, deben realizarse en el EDT evitar errores o que la GUI se congele.



Problemas comunes con hilos en Swing

👉 ***Ver ejemplo: EjemploSwingBloqueo***

- **Bloqueo de la GUI**
  - Ocurre cuando se ejecuta una operación que consume tiempo (por ejemplo, un bucle largo) en un listener. Se bloquea el EDT.
  - Consecuencia: La interfaz gráfica se congela hasta que la operación termine.
- **Actualización de la GUI desde un hilo externo**
  - Modificar componentes de la GUI desde un hilo que no sea el EDT puede causar excepciones o comportamientos inesperados.
  - Solución: Usar `SwingUtilities.invokeLater()`.



Uso de `SwingUtilities`

👉 ***Ver ejemplo: EjemploSwingThread***

- `SwingUtilities.invokeLater(Runnable doRun)` se utiliza para ejecutar código en el EDT. Es útil cuando se necesita actualizar la GUI desde un hilo.
  - Se utiliza comúnmente cuando se realiza una tarea en segundo plano y, una vez terminada, se necesita actualizar la GUI.

```
SwingUtilities.invokeLater(() -> {
    label.setText("Update in EDT");
});
```



Colecciones concurrentes (1/2)

👉 ***Ver ejemplo: ListaConcurrente***

- Collections.synchronizedXXX(), permiten crear versiones sincronizadas de las colecciones estándar de Java (por ejemplo, ArrayList, HashMap, HashSet)
  - Envuelve la colección original en una capa de sincronización que aplica un bloqueo en cada operación (`add()`, `get()`, `remove()`, etc.), de manera que solo un hilo puede acceder a la colección a la vez.
  - Los accesos a las colecciones son bloqueantes como si se utilizase un bloque `synchronized`.



Colecciones concurrentes (2/2)

- El acceso (lectura o escritura) a colecciones tradicionales desde varios hilos de manera simultánea no es seguro.
  - `CopyOnWriteArrayList`: Implementación de `List` que permite concurrencia cuando se realizan modificaciones (`add()` o `set()`).
  - `ConcurrentHashMap`: Implementación de `Map` diseñada para soportar operaciones de lectura y escritura de forma concurrente.
  - `CopyOnWriteArraySet`: Implementación concurrente de `Set`.



Variables `Atomic`

👉 ***Ver ejemplo: EjemploAtomicInteger***

- Permiten realizar operaciones atómicas sobre variables, como incrementos y comparaciones.
- Garantizan que las operaciones se realicen de manera atómica sin usar synchronized.
  - `AtomicInteger`
  - `AtomicLong`
  - `AtomicIntegerArray`



Coordinación de hilos

👉 ***Ver ejemplos del paquete parallel***

- `ExecutorService`: Gestiona un grupo de hilos que se reutilizan para ejecutar varias tareas.
- `CompletableFuture`: Permite ejecutar tareas de forma asíncrona y combinarlas, permitiendo realizar acciones cuando todas las tareas han finalizado.
- `CountDownLatch`: Permite que un grupo de hilos espere hasta que todos los hilos hayan terminado.